Память Go, GC и pprof

время жизни → эскейп-анализ → аллокации → GC → давление памяти → pprof

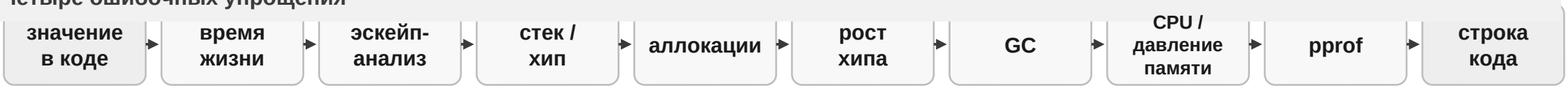
---

Основная лекция • Go 1.27.x

Цель: проследить путь от Go-кода до CPU, RSS и p99

Главная цепочка лекции

Четыре ошибочных упрощения



Лекция строится вокруг времени жизни: где данные живут, кто их удерживает и какую работу выполняют компилятор и рантайм.

«указатель = хип»	«GC съел память»	«короткая пауза = дешевый GC»	«рост RSS = утечка»

Вместо ярлыка — механизм → метрика → профиль → участок кода.

Backend работает с реальной памятью

```
func handler(w http.ResponseWriter,
             r *http.Request) {
    user := User{ID: 42, Name: "Alice"}
    data, _ := json.Marshal(user)
    w.Write(data)
}
```

- стеки горутин
- объекты хипа
- метаданные рантайма
- стеки потоков ОС
- сетевые буферы
- отображения памяти

CPU = 95%

RSS = 1.8 GB

p99 = 900 ms

горутин = 40 000

Синтаксис обработчика не объясняет симптомы под нагрузкой. Важно, как долго живут данные и как рантайм управляет памятью.

Стек: время жизни следует структуре вызовов



Стек дешев благодаря структурированному времени жизни: память освобождается вместе с кадром функции.

Стек горутины: свой, растущий, перемещаемый

G1 → стек 1

G2 → стек 2

G3 → стек 3

У каждой горутины есть собственный стек.

проверка границ стека

места не хватает

рантайм выделяет больший стек

копирует кадры и поправляет известные указатели

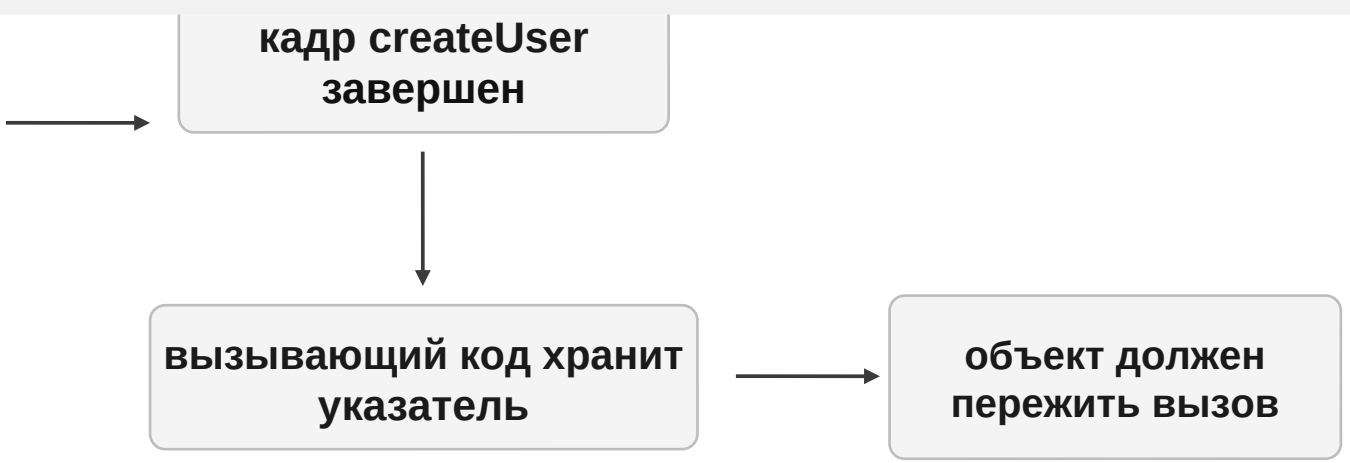
горутина продолжает работу

Много горутин → память стеков + корни GC и удерживаемый хип через ссылки.

Хип: время жизни определяется достижимостью

Полезная модель

```
func createUser() *User {
    u := User{ID: 42}
    return &u
}
```



стек
время жизни следует вызовам

хип
время жизни зависит от достижимости

Стек или хип определяет компилятор, не синтаксис

```
func f() int {
    p := new(int)
    *p = 42
    return *p
}
```

new(T) не гарантирует хип

```
func g() int {
    x := 42
    p := &x
    return *p
}
```

&x не означает хип

```
func counter() func() int {
    n := 0
    return func() int {
        n++
        return n
    }
}
```

локальная переменная может жить дольше вызова

Главный вопрос: сколько должно жить значение и способен ли компилятор доказать короткое время жизни?

Эскейп-анализ: куда уходят ссылки



- возврат указателя
- захват замыканием
- сохранение в долгоживущем объекте
- горутина живет независимо
- непрозрачный косвенный вызов
- большой объект / пределы компилятора

Корректность важнее экономии памяти стека

Если компилятор не доказал короткое время жизни, он выбирает безопасное размещение.

Эскейп-анализ: неочевидные случаи

горутина
может пережить вызывающий код

возвращенный срез
базовый массив переживает вызов

интерфейс / косвенный вызов
может скрыть время жизни

крупный локальный объект
может уйти в хип из-за ограничений реализации

Инлайнинг меняет контекст анализа

до инлайнинга
вызывающий код видит границу

→

тело встроено в вызов

→

компилятор доказывает больше
→ аллокации могут исчезнуть

Число аллокаций зависит от кода и версии компилятора.

Эскейп-анализ: как расследовать аллокации

```
go build -gcflags="-m=2" ./...
```

x уходит в хип

перенос в хип: x

Диагностика компилятора не отвечает, является ли это узким местом.



-m объясняет размещение в памяти.
-benchmem показывает реальные аллокации.
pprof показывает, где они стали дорогими.

Аллокации: три разные величины

Число аллокаций
сколько раз выделяли память

Выделенные байты
сколько прошло через аллокатор

Живые байты
сколько памяти удерживается

Они могут радикально расходиться

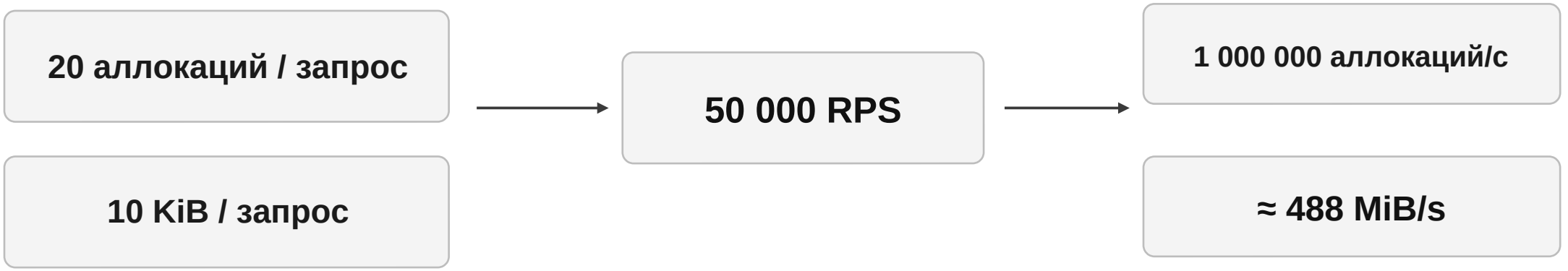
живой хип = 200 MB

аллокаций = 5 GB/s

RSS ≈ стабилен

Малый живой хип может сочетаться с огромным потоком аллокаций и высоким CPU GC.

Поток аллокаций растет вместе с RPS



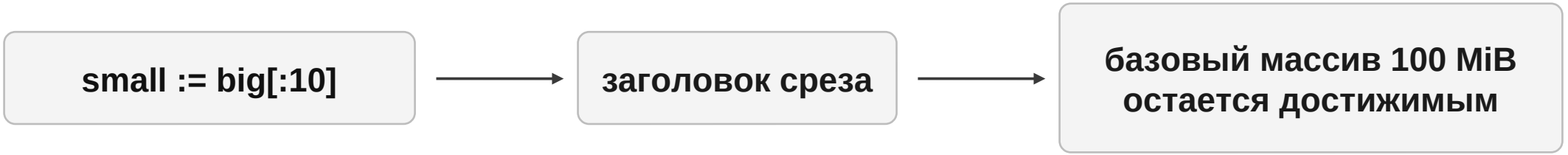
Оптимизация начинается с частоты × стоимости, а не со слов «аллокация маленькая».

Что вызывает аллокации

make / базовые массивы	**рост через append**	**string ↔ []byte**
срезы, map, память	новый массив + копия	часто новая память и копия
fmt / сериализация	**замыкания**	**JSON**
временные буферы / интерфейсы	захваченные данные	DTO + буферы + копии

Нельзя утверждать, что конструкция X всегда выделяет память. Проверяем конкретный путь через бенчмарк и эскейп-анализ.

Владение и удержание важнее самого факта аллокации



Другие владельцы

map
корень → map → значение

context
долгоживущий владелец → ctx → граф

замыкание
захваченные ссылки

очередь / канал
буфер удерживает объекты

Иногда дополнительное копирование небольшого фрагмента снижает давление памяти, потому что большой базовый массив перестает удерживаться.

Указатели: время жизни, алиасинг и локальность

```
a := &User{Name: "Alice"}
b := a
b.Name = "Bob"
```

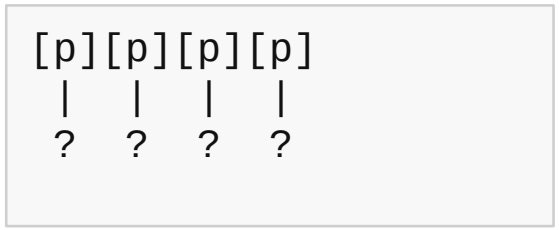
Два указателя → одна область памяти → совместное изменение

[]Point

```
[X Y] [X Y]
[X Y] [X Y]
```

смежные значения → локальность

[]*Point



косвенный доступ + ссылки GC + промахи кеша

Указатель может уменьшить копирование, но добавляет алиасинг, продлевает время жизни объекта и иногда ухудшает локальность.

Размер хипа не равен объему работы GC

Что измерять

```
buf := make([]byte, 100<<20)
```

```
nodes := make([]*Node, n)
```

100 MiB данных
почти без Go-указателей

→

100 MiB массива с указателями
миллионы возможных ссылок

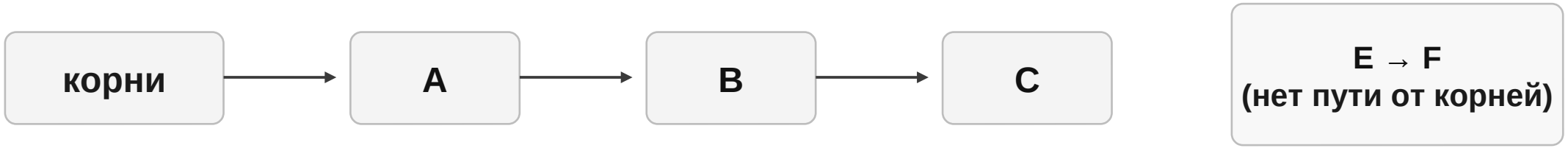
Одинаковый размер → разная
цена сканирования.

/gc/scan/heap:bytes

/gc/scan/stack:bytes

объекты, байты, аллокации в секунду

Сборщик мусора: достижимость, не бизнес-смысл



достижимые → живые

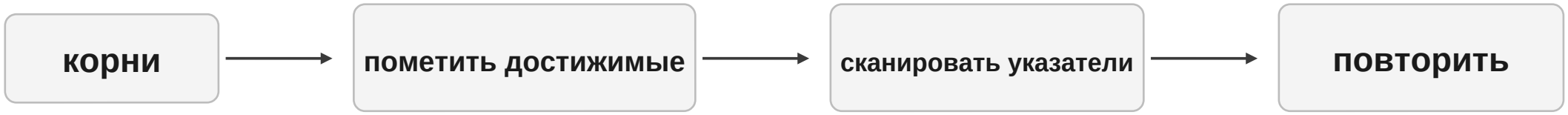
недостижимые → можно удалить

```
var cache = map[string][]byte{}
cache[id] = hugeData
```

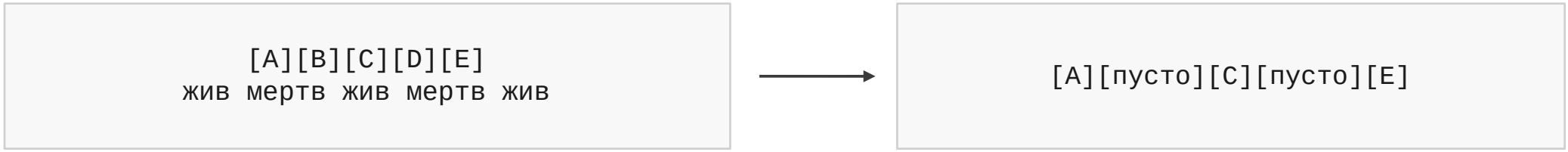
глобальный корень → map → запись → []byte
GC обязан сохранить

Трассирующий GC: маркировка и очистка

Маркировка



Очистка



Три цвета — логическая модель маркировки: белый → серый → черный. Это не поле Color в каждом объекте.

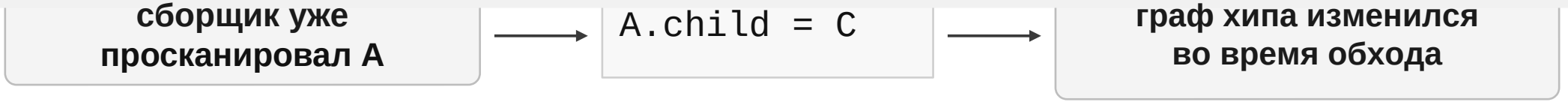
Какой сборщик мусора использует Go

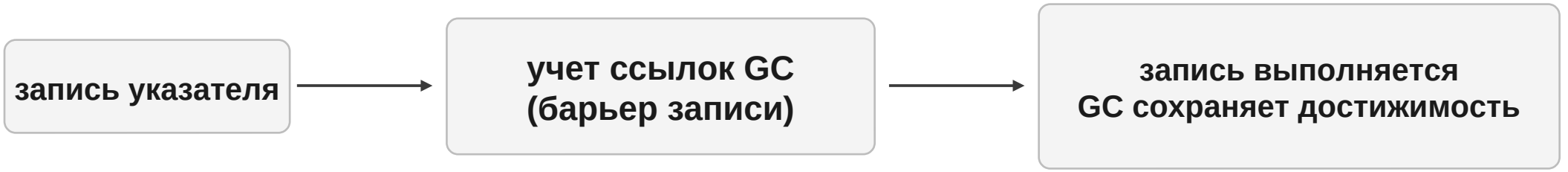
трассирующий	идем от корней по графу
точный	рантайм знает Go-указатели
конкурентный	GC и приложение работают одновременно
параллельный	несколько работников GC
маркировка и очистка	пометить живое → очистить
без поколений	нет разделения на молодой и старый хип
без уплотнения	живые объекты не сдвигаются при уплотнении	

Компромисс Go: короткие паузы STW и конкурентная маркировка за счет CPU рантайма и возможной фрагментации.

Конкурентному GC нужен барьер записи

Решение





Конкурентный GC сокращает паузы, но добавляет фоновую работу, барьеры записи, помощь GC и давление на кеши и память.

Green Tea: улучшаем локальность маркировки

С Go 1.26 это сборщик по умолчанию; модель достижимости прежняя.

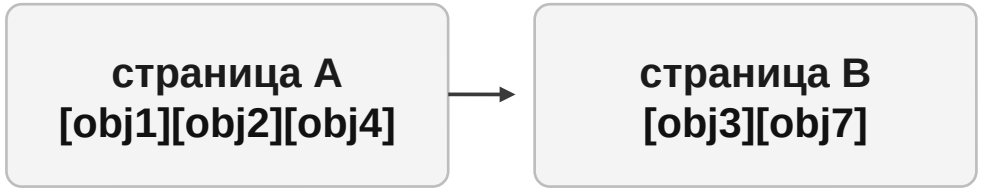
Обход графа по ссылкам



страница 10 страница 7000 страница 53

переходы по ссылкам → промахи кеша

Green Tea



работа по страницам

→ пакетная обработка

→ лучше локальность кеша

→ меньше очереди задач

Green Tea уменьшает цену маркировки и сканирования, но не устраняет поток аллокаций, утечки и рост очередей.

Цикл GC: последовательность этапов, не одна пауза



Конкурентная маркировка включает работников GC, сканирование стеков, барьеры записи и помощь GC.

длительность цикла GC **≠** **длительность паузы STW**

GOGC: компромисс между памятью и CPU

Учебная модель:

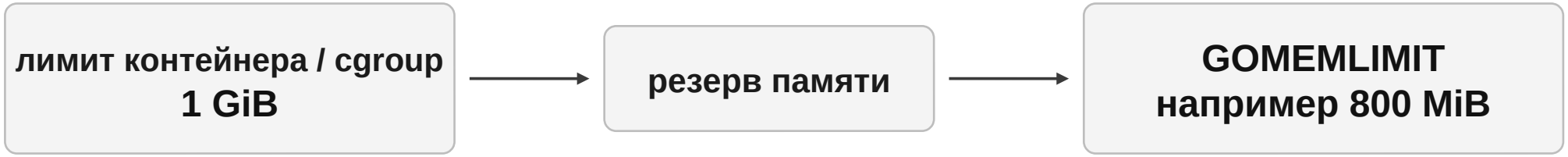
```
цель ≈ живой хип × (1 + GOGC/100)
```

Рантайм учитывает корни, pacer и лимит памяти.

GOGC	50	100	200
рост хипа бюджет	меньше	по умолчанию	больше
частота GC	выше	средняя	ниже
GC CPU	обычно выше	базовый	обычно ниже
память	меньше	базовый	больше

GOGC не сокращает аллокации: он определяет допустимый рост памяти между циклами GC.

GOMEMLIMIT — мягкий лимит; cgroup — жесткий



Почему лимит мягкий?



Запас нужен для рантайма, стеков, cgo, mmap, фрагментации и всплесков нагрузки.

Маленькая пауза GC не означает дешевый GC

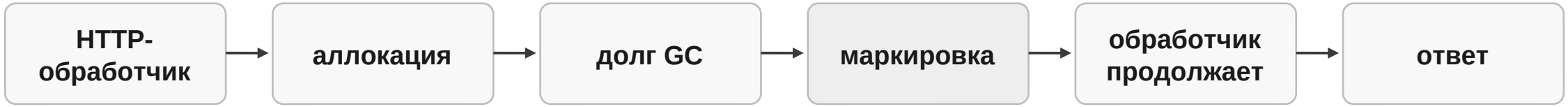
STW = 150 µs → **выглядит отлично**

Но работа GC остается в другом месте

- **фоновый GC CPU**
- **барьеры записи**
- **сканирование стеков**
- **помощь GC**
- **нагрузка на кеш / память**

В эксплуатации смотрим частоту GC, CPU, скорость аллокаций, живой хип, помощь GC и p99.

Помощь GC: работа сборщика внутри запроса

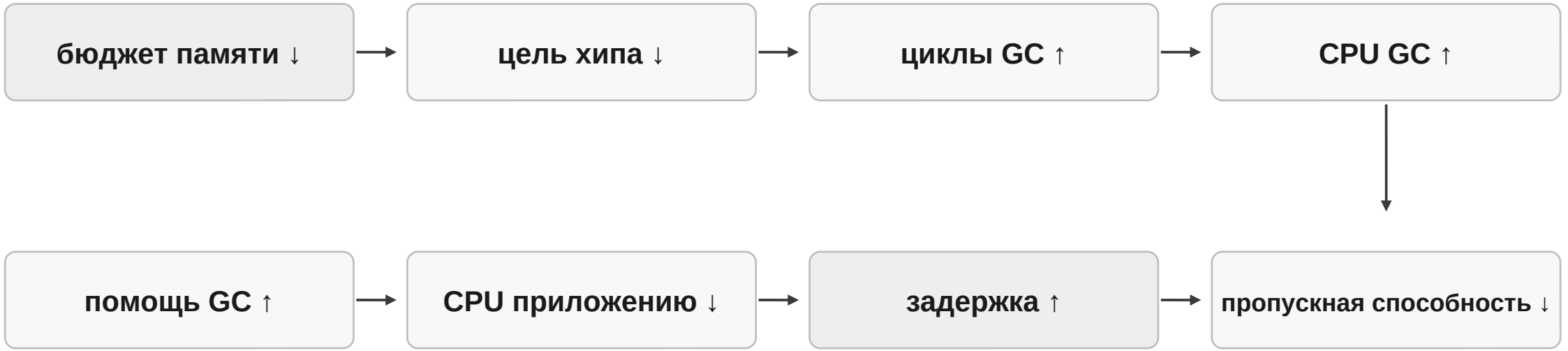


Если фоновые работники GC не успевают, аллоцирующая горутина помогает маркировке — растет время запроса.



помощь GC распределяется неравномерно

Давление памяти превращается в нагрузку на CPU



Давление памяти не обязательно означает утечку: причина может быть в потоке аллокаций, удержании данных, горутинах, очередях или тесном лимите.

Задержка сама увеличивает потребление памяти

```
в работе ≈ RPS × время ответа
```

10 000 RPS
20 ms → **≈ 200 запросов**

10 000 RPS
500 ms → **≈ 5 000 запросов**

Если запрос удерживает 50

200 × 50 KiB ≈ 10 MiB

5 000 × 50 KiB ≈ 244 MiB

Замедление БД или сети увеличивает живой хип даже без роста allocs/op.

Хип, RSS и виртуальное адресное пространство — не одно и то же

Память рантайма Go

- объекты хипа
- свободный хип
- возвращено ОС
- стеки
- метаданные рантайма

Память процесса ОС

- виртуальное адресное пространство
- RSS
- отображенные страницы
- ошибки страниц

Учет контейнера

- учет cgroup
- жесткий лимит памяти
- давление на хост

HeapAlloc снизился, но RSS остался высоким: память свободна для Go, однако физические страницы еще не возвращены ОС.

Утечка памяти при GC: объект удерживается слишком долго

Во всех случаях GC исправен

- неограниченный кеш
- срез удерживает большой базовый массив
- растущая очередь
- утечка горутин



Такая утечка — проблема времени жизни, владения данными и ограничения нагрузки.

pprof: выбираем профиль по симптому

Симптом	Первый инструмент	Что ищем
CPU высокий	профиль CPU	где расходуется CPU
растет живой хип	хип / inuse_space	кто удерживает память
GC CPU высокий, хип стабилен	CPU + alloc_space + метрики	источник аллокаций
горутин становится больше	профиль горутин	где и на чем блокируются
p99 высокий, CPU низкий	трассировка + метрики зависимостей	ожидание БД / сети / очередей
HeapAlloc упал, RSS нет	метрики Go + OC / cgroup	свободная / внешняя память

pprof CPU: flat, cum и цепочка вызовов

```
go tool pprof \
 'http://localhost:6060/debug/pprof/profile?seconds=30'
```

flat
CPU внутри функции

cum
функция и ее вызовы

top -cum
источник
дорогих вызовов

list
конкретные строки
исходника

```
p99 = 5s
CPU = 20%
```



БД / сеть / пул / блокировка

Ожидание не тратит CPU. Медленный обработчик может почти не попасть в профиль CPU.

pprof heap: поток аллокаций и удержание

Тип выборки	Вопрос	Типовая задача
inuse_space	кто удерживает память?	удержание / утечка / большой хип
inuse_objects	кто удерживает объекты?	много объектов / нагрузка GC
alloc_space	кто выделил память?	поток аллокаций / нагрузка GC
alloc_objects	кто создал объекты?	поток мелких краткоживущих объектов

Один снимок не доказывает утечку. Сравниваем профили во времени: t0 → t1 → t2.

pprof горутин: число и стеки ожидания

Характерные признаки

горутин = 20 000 → **норма? утечка?** → **смотрим основные стеки**

```
15 000 горутин
→ одинаковый стек
→ ожидание в канале получение
```

возможная утечка

```
10 000 горутин
→ database/sql
→ ожидание соединения
```

пул / БД — первопричина надо расследовать

Go 1.27: goroutineleak находит автоматически ограниченный класс постоянных утечек

runtime/metrics: метрики рядом с pprof

Состояние хипа

```
/gc/heap/live:bytes
/gc/heap/goal:bytes
/gc/heap/allocs:bytes
/gc/heap/frees:bytes
```

Сканирование

```
/gc/scan/heap:bytes
/gc/scan/stack:bytes
```

GC CPU

```
mark/assist
mark/dedicated
mark/idle
gc/total
```

Классы памяти

```
heap/objects
heap/free
heap/released
```

Метрики показывают динамику. pprof связывает нагрузку с кодом. Трассировка нужна для последовательности событий.

Под нагрузкой: высокая аллокационная активность



После исправления



Узкое место GC часто возникает из-за интенсивных аллокаций приложения, а не из-за настройки сборщика.

Под нагрузкой: задержка зависимости → давление памяти

Первопричина



Все объекты запросов достижимы. GC работает корректно: он не имеет права их удалить.

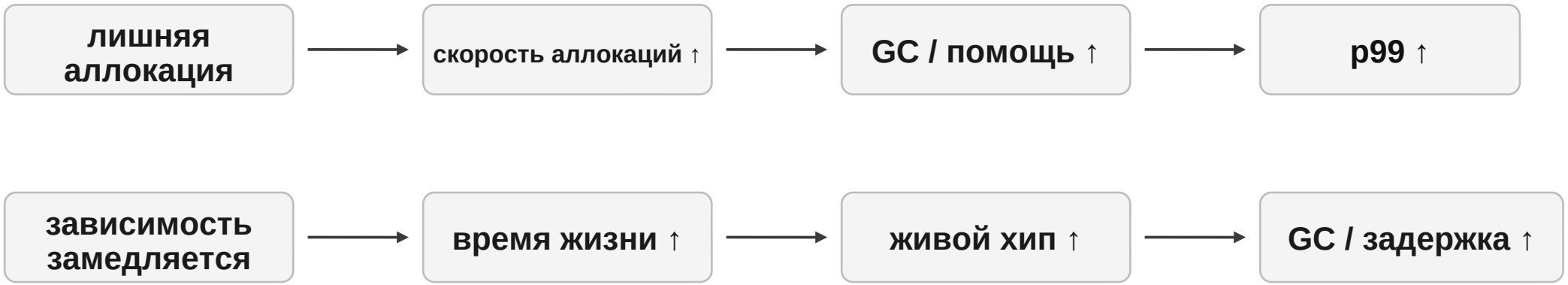
задержка зависимости / слабое ограничение нагрузки

Исправлять БД, пул соединений, таймауты, ограничения параллелизма и очереди. Начинать с GOGC=400 — плохая идея.

Итоговая схема: от кода к диагностике



Две цепочки в эксплуатации



Главный вывод: работа с памятью начинается с времени жизни и аллокаций, а заканчивается измерением конкретного участка кода.